

ASSIGNMENT

Assessment 5

Opened: Thursday, 20 August 2026, 8:00 AM

Due: Friday, 11 September 2026, 11:00 PM

159383

Final Practical Project

COS2611 Final Practical Project 2026

Municipal Incident Response System

1. Project Overview

Municipalities and local authorities across Southern Africa are responsible for responding to incidents that affect communities. These incidents may include problems relating to water, electricity, roads, sanitation and public facilities.

When several incidents are reported at the same time, an operations centre needs to decide which incidents require attention, which response teams are available, and how a response team can reach the affected location.

For this project, you have been asked to develop a **Municipal Incident Response System** for a fictional municipal operations region in **Southern Africa**.

The region consists of real Southern African locations connected by roads. A limited number of response teams are available to deal with reported incidents.

Your task is to design and implement a C++ program that assists the municipality in managing and responding to these incidents.

Important

There is **more than one possible way to design a solution** to this problem. You are expected to analyse the problem, decide how the information should be represented and determine appropriate ways of processing the information.

You are **not** being given a prescribed data structure or algorithm for solving the problem.

2. Project Scenario

The municipal operations centre receives reports of incidents occurring at different locations within its area of responsibility.

Each incident has a type and severity, and each incident occurs at one of the locations in the region.

The municipality has a limited number of response teams. Each team has a particular capability and is currently located at one of the locations in the region.

The locations are connected by roads, and the roads have associated travel times.

The operations centre therefore needs a system that can help it answer questions such as:

- What incidents are currently open?
- Which incidents should receive attention first?

?

- Which response teams are suitable for a particular incident?
- How can a response team reach an incident location?
- What is the current status of an incident or response team?

Your program must provide functionality that assists the operator with these types of decisions.

3. Locations

The system must contain **10 real locations in Southern Africa**.

You must select the 10 locations that will form part of your scenario.

The locations may be selected from South Africa or neighbouring Southern African countries. However, the selected locations should form a **plausible geographical region** for the purposes of the project.

Each location must have:

- a unique identifier;
- a real location name.

The names of the selected locations must be **hard-coded in your program**.

You may **not** invent locations for the purpose of satisfying this requirement.

Real-world data

You must conduct appropriate research to identify your selected locations and the road connections between them.

The sources used to establish your locations and road connections must be recorded in comments in your C++ source file.

For example:

```
// Real-world data sources:  
// Source 1: ...  
// Source 2: ...
```

You do not need to submit separate source documents.

4. Road Network

The selected locations must be connected by between **15 and 20 road connections**.

The connections must represent **realistic, existing road connections** between the selected locations.

You may not simply invent connections between locations that are not connected by a suitable road route.

The road connections must be **hard-coded in your program**.

Each connection must have a travel time between **5 and 60 minutes**.

The travel times should be based on realistic real-world information or reasonable estimates derived from real-world information.

The complete network must be connected. This means that every selected location must be reachable from the other locations through one or more road connections.

You must decide how the road network should be represented and processed in your program.

You are not instructed to use a particular graph representation or path-finding algorithm.

5. Response Teams

The system must generate **6 response teams** when the program starts.

Each response team must have:

- a unique identifier;
- a response capability;
- a current location;
- a current status.

Possible response capabilities include:

- Water
- Electricity
- Roads
- Sanitation
- Public Facilities

You may include other suitable capabilities if you believe they are appropriate for your design.

Response teams may have the following statuses:

- **Available**
- **Assigned**
- **Unavailable**

At the beginning of the program, **at least three response teams must be available**.

The response-team information must be generated programmatically.

6. Incidents

The system must generate **10 incidents** when the program starts.

Each incident must have:

- a unique incident number;
- a location;
- an incident category;
- a severity level;
- a status.

The incident categories are:

1. Water
2. Electricity
3. Roads
4. Sanitation
5. Public Facilities

The severity levels are:

- Low
- Medium
- High

All incidents must initially have an **Open** status.

Incident information must be generated programmatically.

7. Randomly Generated Information

The project contains both **fixed geographical information** and **randomly generated operational information**.

The following information must be hard-coded:

- the 10 selected locations;

- the names of the locations;
- the 15–20 road connections;
- the travel times associated with the road connections;
- the random-number seed.

The following information must be generated programmatically:

- incident locations;
- incident categories;
- incident severity levels;
- response-team capabilities;
- response-team starting locations;
- response-team starting statuses.

The purpose of the random generation is to create a changing operational situation while keeping the underlying geographical environment consistent.

Your program must not depend on a fixed set of manually entered incidents or response-team information.

8. Random-Number Seed

The random-number seed must be **hard-coded in your program**.

For example:

```
const unsigned int SEED = 2611;
```

The seed must be used to generate the operational information.

Using the same seed must reproduce the same operational situation.

You may use a different seed while developing and testing your program. However, the submitted program must contain a clearly identifiable hard-coded seed value.

The marker may change the seed during testing to determine whether your solution continues to work correctly with different generated operational situations.

9. Validity of Generated Information

The generated information must be logically valid.

For example:

- an incident must refer to one of the selected locations;
- a response team must have a valid location;
- a response team must have a valid capability;
- generated team statuses must be valid;
- all locations must form part of the connected road network;
- at least three response teams must initially be available.

Your program should continue to operate correctly when the generated operational information changes.

10. Suggested Menu Structure

Your program must provide a menu through which the operator can access the functionality of the system.

The following is a **suggested menu structure**. You may organise your menu differently if your design provides the required functionality.

```

=====
MUNICIPAL INCIDENT RESPONSE SYSTEM
=====

1. Display Locations and Roads
2. Display Incidents
3. Prioritise Open Incidents
4. Display Response Teams
5. Investigate Incident
6. Find Route
7. Update Incident
8. Display Summary
0. Exit

Enter your choice:

```

Your menu should be clear and easy for an operator to use.

Option 1: Display Locations and Roads

Display the locations in the system and the road connections between them, including relevant travel-time information.

Option 2: Display Incidents

Display the current incidents and their relevant information.

The information should allow the operator to identify which incidents are open and understand their severity and location.

Option 3: Display Response Teams

Display the response teams, their capabilities, locations and current status.

Option 4: Investigate Incident

Allow the operator to select an incident and obtain useful information about it.

The system should assist the operator in identifying suitable available response teams for the selected incident.

You must decide how the system determines which teams are appropriate.

Option 5: Find Route

Allow the operator to select two locations and obtain useful information about travelling between them.

The system should provide an appropriate route through the available road connections and relevant travel information.

You must decide how this functionality is implemented.

Option 6: Update Incident

Allow the operator to update an incident as it is processed.

For example, an incident may change from **Open** to **Assigned** and eventually to **Resolved**.

If a response team is assigned to an incident, the team's status should also be updated appropriately.

Option 7: Display Summary

Display a useful summary of the current operational situation.

For example, the summary could include:

- number of open incidents;
- number of assigned incidents;
- number of resolved incidents;
- number of available teams;
- number of unavailable teams.

You may include other useful information.

11. Incident Prioritisation

The operations centre needs to decide which outstanding incidents should receive attention first.

Your program must provide a meaningful way of **prioritising open incidents**.

You are **not** given a prescribed prioritisation formula.

You must consider the information available about an incident and develop a sensible approach to prioritisation.

Your approach should be logical, consistently applied and useful to the operations centre.

The approach you choose is part of your solution design.

12. Response Team Selection

When an incident requires attention, the operations centre needs to identify a suitable response team.

Your system must assist the operator in identifying an appropriate available response team.

Relevant information may include:

- the team's capability;
- the team's current location;
- the incident location;
- the team's availability;
- travel requirements.

You must decide how your system uses this information.

You are **not** given a prescribed method for selecting a response team.

13. Travel and Routes

The operations centre must be able to investigate how locations within the region are connected.

Your program must provide useful information about travelling between selected locations.

You must decide how to determine a suitable route using the available road connections.

You are **not** instructed to use a particular graph representation or path-finding algorithm.

14. Updating the Situation

The operational situation must be capable of changing while the program is running.

For example, when an incident is assigned:

- the incident status should change;
- the assigned response team should no longer be treated as available.

When an incident is resolved:

- the incident status should change to Resolved;
- the response team should become available again.

The program should maintain a consistent operational state as changes are made.

15. Solution Design

A central purpose of this project is to assess your ability to **analyse a practical problem and design a programming solution**.

You are therefore not given a prescribed implementation.

The project does **not** require you to use a particular:

- data structure;
- STL container;
- searching algorithm;
- sorting algorithm;
- graph representation;
- path-finding algorithm;
- class design;
- function structure.

You must decide which approaches are appropriate for your solution.

Different students may therefore produce different solutions while still meeting the requirements of the project.

Your choices should be appropriate to the problem and should result in a program that is:

- functional;
- understandable;
- logically structured;
- reasonably efficient;
- able to work with different generated operational information.

16. Technical Requirements

Your submission must comply with the following requirements:

1. The complete solution must be contained in **one .cpp source file**.
2. Only **standard C++ libraries** may be used.
3. No external libraries or frameworks may be used.
4. No database may be used.
5. No external data files may be used.
6. All operational data must be generated by the program.
7. The program must be a console-based application.
8. The program must compile and execute correctly using the C++ compiler specified for COS2611.
9. The program must not depend on hard-coded operational results for a particular generated scenario.
10. The program should be appropriately divided into functions.
11. Input should be validated where appropriate.
12. The program should handle unsuitable user input without terminating unexpectedly.
13. **You may NOT use genAI to develop the code.**

17. Real-World Data and Sources

Although the incidents and response-team information are randomly generated, the **10 selected locations and road connections must be based on real Southern African information**.

You must conduct appropriate research before implementing your program.

Your source comments should identify the sources used for:

- the selected locations;
- the road connections;
- relevant travel-time information, where applicable.

The generated incidents and response teams do not need to represent actual real-world events. They form part of the simulated municipal environment.

18. Programming Approach

You are expected to apply the programming knowledge acquired throughout COS2611.

The quality of your solution will depend not only on whether the program produces output, but also on whether your chosen approach is appropriate for the problem.

You should consider:

- how information should be represented;
- how information should be organised;
- how information should be searched;
- how information should be processed;
- how incidents should be prioritised;
- how suitable response teams should be identified;
- how routes should be determined;
- how the system should respond when the operational situation changes.

You should avoid unnecessary complexity.

The objective is to produce a **well-designed solution to the given problem**, not to include as many programming techniques as possible.

19. Important Notice About the Solution

There is no single correct implementation of this project. Do not use genAI in the development of the solution.

You are being assessed on your ability to:

Understand the problem → Design a solution → Implement the solution → Test the solution

You should not attempt to guess a particular data structure or algorithm that the lecturer expects.

If you choose a particular approach, you should be able to explain why that approach is appropriate for the problem. A simple approach that is appropriate and correctly implemented may therefore be better than a complicated solution that does not meaningfully improve the system.

20. Submission Requirements

Submit **exactly one C++ source file** containing your complete solution.

No additional source files, libraries, databases or external data files may be submitted as part of the solution.

The source file must contain:

- the hard-coded real-world locations;
- the hard-coded road connections;
- the hard-coded travel times;
- the hard-coded random-number seed;
- comments identifying the real-world sources used for the geographical information.

21. Solution Reflection

At the **END of your C++ source code**, after your complete program, include comments containing a short reflection on how you arrived at your solution.

Your reflection should explain the **thinking and design decisions** that led to your solution. This is not a description of what each line of code does.

In your reflection, discuss:

- How you analysed the problem before starting to code;
- How you decided to represent the locations, roads, incidents and response teams;

- Why you chose the particular data structures and algorithms used in your solution;
- How you approached the problem of prioritising incidents;
- How you approached identifying a suitable response team;
- How you approached finding routes between locations;
- Any significant design decisions or challenges you encountered;
- How you tested your solution and what you did when you encountered problems.

Your reflection should be written in your own words and should demonstrate that you understand the solution you have implemented.

Important: The reflection forms part of your submission and may be considered when evaluating whether you understand and can explain the solution you developed.

Example format at the end of your source file:

```
// =====
// SOLUTION REFLECTION
// =====
//
// I first analysed the problem by ...
//
// I decided to represent ...
// because ...
//
// For the route-finding problem, I chose ...
// because ...
//
// One challenge I encountered was ...
// I solved this by ...
//
// I tested my program by ...
// =====
```

IMPORTANT: REQUIRED FILENAME

Your submitted C++ file **must be named exactly:**

studentNumber_FPP.cpp

For example, if your student number is **12345678**, your file must be named:

12345678_FPP.cpp

The student number in the filename must be your own correct student number.

The following will result in a **ZERO MARK** for the Final Practical Project:

- the student number is missing from the filename;
- the student number in the filename is incorrect;
- the filename does not follow the required format;
- the submitted file cannot be linked to the correct student in the Moodle system.

This requirement is essential because the marking process uses the student number in the filename to link the submitted program to the corresponding student record in Moodle.

DO NOT USE ANY OTHER FILENAME.

Your submission must contain exactly one C++ source file named:

studentNumber_FPP.cpp

22. Final Advice

You have **two weeks** to complete this project.

Do not begin by trying to write the entire program at once.

First understand the problem and decide how you will represent the information.

Then develop your solution incrementally and test each part before adding the next.

Remember that the purpose of the project is not to reproduce an example from the study material. You are being asked to take a practical problem, make reasonable design decisions and develop your own working solution.

There is more than one way to solve the problem.

Your solution should demonstrate that you can use C++ to turn a practical problem into a functioning computer-based solution.

COS2611 Final Practical Project 2026

One problem. One .cpp file. Two weeks. Your solution design.

COS2611 Final Practical Project 2026 – Assessment Rubric

Total: 100 marks

The project will be assessed primarily on the **functionality and correctness** of the submitted program. Different valid design approaches may be used. You will not receive additional marks simply for using a particular data structure or algorithm.

1. Compilation and Program Execution – 10 marks

- **8–10 marks:** Program compiles and executes correctly. The program starts correctly and provides a functional menu/interface through which the required functionality can be accessed.
- **5–7 marks:** Program compiles and executes but has some significant functionality or interface problems.
- **1–4 marks:** Program has major execution problems or only limited functionality can be demonstrated.
- **0 marks:** Program does not compile or cannot be executed.

2. Locations and Road Network – 15 marks

- **5 marks:** 10 real Southern African locations are correctly included and hard-coded.
- **3 marks:** Between 15 and 20 road connections are correctly included and hard-coded.
- **2 marks:** Travel times are provided and fall within the required range.
- **2 marks:** Road connections refer to valid locations.
- **3 marks:** The road network is logically connected and locations can be reached through the network.

3. Random Data Generation – 10 marks

- **2 marks:** 10 incidents are generated programmatically.
- **2 marks:** 6 response teams are generated programmatically.
- **2 marks:** Incident information, including location, category and severity, is generated appropriately.
- **2 marks:** Response-team information, including capability, location and status, is generated appropriately.
- **1 mark:** A hard-coded random-number seed is used.
- **1 mark:** The generated information is not simply a fixed, hard-coded operational scenario.

4. Incident Management – 15 marks

- **4 marks:** Incidents can be displayed and investigated.
- **4 marks:** Incidents are processed using their relevant information, including location, category and severity.
- **4 marks:** The program provides a meaningful mechanism for prioritising incidents.
- **3 marks:** Incident status can be updated correctly as the incident is processed.

5. Response Team Management – 15 marks

- **4 marks:** Response teams have appropriate capabilities, locations and statuses.
- **4 marks:** The program can identify response teams that are suitable for an incident.
- **4 marks:** Team location and/or travel considerations are appropriately incorporated when relevant.
- **3 marks:** Team status is updated correctly when a team is assigned or released.

6. Route Finding – 15 marks

- **4 marks:** The user can select or specify locations for which a route is required.
- **5 marks:** The program determines a valid route through the available road network.
- **3 marks:** Relevant travel information, such as total travel time, is provided.
- **3 marks:** The route-finding solution works correctly for different locations and does not depend on hard-coded results for particular locations.

7. Program Design and C++ Implementation – 10 marks

- **3 marks:** The program is appropriately divided into functions.
- **3 marks:** Data structures and C++ features are used appropriately for the problem.
- **2 marks:** The program is logically organised and reasonably easy to understand.
- **2 marks:** The solution avoids unnecessary complexity and demonstrates sensible design decisions.

8. Input Handling and Robustness – 5 marks

- **3 marks:** Invalid or unsuitable user input is handled appropriately.
- **2 marks:** The program does not crash or enter an unusable state during normal operation and continues to maintain a valid operational state.

9. Solution Reflection – 5 marks

- **2 marks:** A reflection is included at the end of the source code.
- **1 mark:** The reflection explains how the student approached and analysed the problem.
- **1 mark:** The reflection explains important design decisions, including data structures and/or algorithms used.
- **1 mark:** The reflection discusses testing, challenges encountered or lessons learned.

Marking Principles that will be applied by the marker:

Different valid solutions are expected.

Students are not required to use a particular data structure, STL container, searching algorithm, sorting algorithm, graph representation or path-finding algorithm.

Marks should therefore be awarded based on whether the student's chosen approach **correctly and effectively solves the required problem**.

A simple solution that correctly solves the problem should not be penalised merely because another student used a more sophisticated technique.

Administrative Requirements

Filename compliance is a compulsory submission requirement.

The submitted file must be named:

studentNumber_FPP.cpp

If the student number is missing from the filename, incorrect, or the filename does not follow the required format, the submission will receive a **ZERO MARK**.

This is necessary because the student number in the filename is used to link the submitted program to the correct student record in Moodle.

Mark Allocation Summary

Assessment Criterion	Marks
1. Compilation and Program Execution	10
2. Locations and Road Network	15
3. Random Data Generation	10
4. Incident Management	15
5. Response Team Management	15
6. Route Finding	15
7. Program Design and C++ Implementation	10
8. Input Handling and Robustness	5
9. Solution Reflection	5
TOTAL	100

[Add submission](#)

Submission status

Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	18 days 12 hours remaining
Last modified	-
Submission comments	▶ Comments (0)

Contact us



Follow us



You are logged in as ROWAN VAN ZYL (Log out)

Data retention summary

Get the mobile app

Get the mobile app

This page is: General type: incourse. Context Assignment: Assessment 5 (context id 32468639). Page type mod-assign-view.